

Saúde+ (Lagoa Conectada) — Do Zero ao Produto Comercial

Documento de planejamento de produto, escrito como se o projeto estivesse começando agora, do zero. Serve dois propósitos: (1) ensinar a equipe — adolescentes sem experiência prévia de programação, apoiados por IA — o raciocínio por trás de cada decisão técnica e de produto; (2) servir de roadmap real de Sprint 0 até o lançamento comercial.

Versão original do projeto: proposta da equipe "Conectados para o Sucesso" (programa Passaporte Digital, Lagoa de Itaenga-PE), documento-fonte em [docs/Conectados para o Sucesso 2.0.pdf](#). Aquele documento previa 5 meses corridos, sem sprints, sem papéis de time definidos. Este documento substitui aquele cronograma por um plano de produto real — mantendo o problema, o público e o objetivo originais.

1. Visão geral do produto

1.1 Problema

Moradores de Lagoa de Itaenga (PE) têm dificuldade para marcar consultas na rede municipal de saúde e para conseguir transporte sanitário até hospitais fora do município. O processo hoje é manual, presencial ou por telefone, sem visibilidade de horários, sem histórico centralizado, sem alerta de surtos/doenças por região.

1.2 Público-alvo

- **Paciente:** morador da rede municipal, baixa a média familiaridade digital, frequentemente em zona rural com internet instável.
- **Atendente:** recepcionista/profissional de saúde do posto/hospital, usa o sistema para atender pacientes presencialmente ou por telefone.
- **Admin:** gestor da Secretaria de Saúde, responsável por unidades, alertas de doença e supervisão de uso.

1.3 Proposta de valor

Um único aplicativo web onde o paciente agenda consulta, pede transporte sanitário e recebe alerta de doença por região — sem precisar ligar, ir presencialmente ou depender de papel. Para a prefeitura, uma ferramenta de gestão com histórico, auditoria e dado estruturado (em vez de planilha ou caderno).

1.4 O que muda em relação à proposta original

O documento-fonte (PDF) previa um app móvel genérico com 5 meses de "planejamento → design → programação → testes → lançamento", sem detalhar arquitetura, sem papéis de Scrum, sem plano de comercialização. Este plano:

- Fixa a stack (web, não nativo — mais barato de manter para um time iniciante).
 - Divide o trabalho em sprints Scrum de 2 semanas, com backlog priorizado e Definition of Done por entrega.
 - Assume que o produto **será comercializado** (licenciado à prefeitura, possivelmente replicável para outros municípios) — não é só um projeto de feira de inovação, é software que vai para produção com dado de saúde real de pessoas reais.
 - Reconhece que já existe uma base de código funcional (ver seção 3.5) e usa essa base como ponto de partida real, não como algo a descartar.
-

2. Papéis da equipe (Scrum)

Time pequeno, todos aprendendo a programar com apoio de IA. Os papéis de Scrum são responsabilidades, não cargos separados de pessoas diferentes — em time pequeno, uma pessoa pode acumular mais de um papel, mas os chapéus continuam distintos nas cerimônias.

Papel	Responsabilidade	Observação para este time
Product Owner (PO)	Dono do backlog, prioriza o que entra em cada sprint, representa a voz da prefeitura/usuário final	Deve ser quem mais entende o problema de saúde local (contato mais próximo da Secretaria de Saúde ou do público-alvo)
Scrum Master (SM)	Facilita as cerimônias (planning, daily, review, retro), remove impedimentos, protege o time de escopo excessivo	Papel crítico para time iniciante: sem SM, sprints viram "tentar fazer tudo" e nada fecha
Dev Team	Implementa o backlog	Todos os adolescentes do time, independente de nível de experiência prévia
IA (copiloto)	Não é um papel de Scrum formal, mas é ferramenta declarada de todo o time	Gera código, explica conceitos, revisa. Não substitui entendimento — Definition of Done de cada item exige que pelo menos uma pessoa do time consiga explicar o que o código faz, não só colar o que a IA gerou

2.1 Cerimônias (sprints de 2 semanas)

- **Sprint Planning** (início do sprint, ~1h): PO apresenta prioridade, time estima o que cabe.
- **Daily** (10-15 min, pode ser 2-3x/semana em vez de diário para time extraescolar): o que fiz, o que vou fazer, o que trava.
- **Sprint Review** (fim do sprint, ~30-45 min): demo do que foi entregue, rodando de verdade — não slide.
- **Retrospectiva** (fim do sprint, ~20-30 min): o que funcionou, o que não funcionou, o que muda no próximo sprint.

3. Arquitetura alvo

3.1 Stack

- **Frontend:** HTML5 + CSS3 + JavaScript puro (vanilla), sem framework, sem build step. Decisão deliberada: time iniciante aprende a base da web sem a complexidade extra de React/Vue/bundlers. Todo o HTML/CSS/JS roda direto no navegador.
- **Backend:** Node.js + Express — **também JavaScript**, então a promessa de "uma linguagem do início ao fim" se mantém no sentido prático (mesma sintaxe, mesmo raciocínio, sem trocar de paradigma). O servidor expõe uma API REST em JSON.
- **Banco de dados:** MySQL — dado relacional (usuários, agendamentos, transporte, doenças) com integridade referencial (chaves estrangeiras), essencial para dado de saúde onde inconsistência é inaceitável.
- **Autenticação:** JWT guardado no `localStorage` do navegador (ver trade-off documentado em 3.4).

3.2 Por que não "tudo no navegador, sem servidor"

Foi cogitado manter tudo em HTML/CSS/JS sem nenhum servidor próprio (dado salvo só no navegador do usuário). Descartado porque:

- Dado de saúde precisa ser acessado por mais de uma pessoa (paciente cadastra, atendente vê, admin supervisiona) — armazenamento só local não permite isso.
- Comercialização para prefeitura exige um sistema real, multiusuário, com histórico e auditoria — não um protótipo de navegador único.
- LGPD e dado sensível de saúde exigem controle de acesso do lado do servidor (o navegador do usuário não é confiável para impor regra de segurança).

3.3 Estrutura de pastas (alvo, já em uso)

```
saude-plus/
├─ index.html          # landing page
├─ auth.html           # login e cadastro (wizard 4 passos)
├─ dashboard.html      # painel do paciente
├─ admin.html          # painel do administrador
├─ atendente.html      # painel do atendente
├─ assets/
│   └─ styles.css      # design system (CSS custom properties)
│   └─ api.js          # cliente HTTP + helpers
├─ server/
│   └─ schema.sql      # estrutura MySQL
│   └─ src/
│       └─ app.js       # Express + rotas
│       └─ server.js    # entrypoint
│       └─ db.js        # pool MySQL
│       └─ middleware/auth.js # JWT + checagem de papel
│       └─ routes/      # auth, appointments, transports, exams,
│                           # reminders, diseases, units, admin, staff
└─ scripts/            # seed-admin, migração
```

3.4 Decisões técnicas já tomadas (herdadas, mantidas com justificativa)

- **JWT em localStorage** : simples de implementar, aceitável para o estágio atual. Risco: token vazado por XSS fica válido até expirar. Mitigação planejada na Fase de Segurança (Sprint 1).
- **bcryptjs em vez de bcrypt** : JS puro, sem dependência de compilador nativo — evita falha silenciosa de instalação em máquina de estudante.
- **SQL parametrizado em 100% das queries**: sem risco de SQL injection, já implementado.
- **Isolamento por user_id** : cada paciente só acessa os próprios dados.

3.5 Ponto de partida real (não é ficção "do zero")

Diferente do que o título sugere, este NÃO é um projeto vazio. Já existe uma base funcional, testada ponta a ponta:

- 5 telas HTML funcionando (`index` , `auth` , `dashboard` , `admin` , `atendente`).
- API REST com ~30 endpoints cobrindo autenticação, agendamento de consultas/exames, transporte sanitário, alertas de doença, unidades de saúde, administração de usuários e log de auditoria.
- Banco MySQL com 5+ tabelas relacionadas por chave estrangeira.
- Dois diagnósticos técnicos completos já produzidos: [PLANO MELHORIA.md](#) (backend/segurança/dado) e [PLANO UIUX.md](#) (interface/acessibilidade).

As sprints deste plano tratam a Sprint 0 e 1 como **fundação documentada** (o time entende o que já existe e por quê) e usam os achados dos dois planos acima como **backlog priorizado** a partir da Sprint 2.

4. Product Backlog (épicas)

Ordenado por prioridade de negócio, não por ordem de execução (a ordem de execução está nas sprints, seção 5).

#	Épico	Fonte	Por que importa para comercialização
E1	Fundação segura (segredo fora do código, rate limiting, headers HTTP)	PLANO_MELHORIAS Fase 1	Sem isso, nenhum deploy real é responsável — dado de saúde exposto
E2	Validação de dado ponta a ponta (front + back)	PLANO_MELHORIAS Fase 2	Prefeitura não aceita sistema que grava CPF/data inválidos
E3	Acessibilidade e substituição de <code>alert()/confirm()</code> nativos	PLANO_UIUX Fase 1	Público-alvo tem baixa familiaridade digital — usabilidade é requisito, não luxo
E4	Consolidação de design system e remoção de duplicação (CSS/JS)	PLANO_MELHORIAS #18, PLANO_UIUX Fase 3	Manutenção barata é o que permite o time pequeno sustentar o produto pós-lançamento
E5	Testes automatizados (mínimo: rotas críticas de auth e agendamento)	PLANO_MELHORIAS #8	Sem teste, toda mudança arrisca quebrar login/agendamento sem ninguém perceber
E6	Deploy real (ambiente de produção, HTTPS, variável de ambiente segura)	README "pendências conhecidas"	Pré-requisito literal para existir como produto, não só protótipo local
E7	LGPD e consentimento explícito de dado de saúde	PLANO_MELHORIAS #91 (readme), item 35	Requisito legal para operar com dado de saúde de pessoas reais no Brasil
E8	Lembretes automáticos (job agendado)	README pendência	Funcionalidade prometida na proposta original, ainda não implementada
E9	Multi-tenant leve (permitir outro município usar, se vendido de novo)	Decisão de comercialização (seção 6)	Habilita revenda sem reescrever o sistema por cliente
E10	Onboarding e material de apresentação para gestor público	Comercialização	Prefeitura decide comprar/adotar vendo demonstração, não código
E11	Monitoramento e log estruturado em produção	PLANO_MELHORIAS #24	Sem isso, time não sabe que o sistema caiu até um usuário reclamar
E12	Alinhamento com padrão de dado do SUS (CNS, prioridade legal, nome social, acompanhante)	Análise comparativa contra CADSUS/e-SUS AB (seção 4.1)	Sem CNS o sistema não interoperava com nenhum sistema público de saúde real — bloqueia adoção séria por prefeitura

4.1 Gap de alinhamento com o padrão de dado do SUS

O cadastro atual mistura dado de identidade civil (CPF, RG, nome da mãe) com perfil pessoal (tipo sanguíneo, gênero), mas não usa o identificador que qualquer posto/hospital do SUS usa de fato para abrir atendimento: o **CNS (Cartão Nacional de Saúde)**. Comparação contra o padrão CADSUS/e-SUS AB:

Campo/conceito oficial	Uso real na rede pública	Presente hoje?
CNS (15 dígitos, CADSUS/CNES)	Identificador único de paciente em e-SUS AB, SISREG (marcação de consulta), TFD (equivalente ao módulo de transporte sanitário do app)	Não
Nome social	Obrigatório desde Portaria nº 1.820/2009	Não
CID-10 / motivo codificado	Toda consulta/exame do SUS é vinculada a CID, não texto livre — base de vigilância epidemiológica, conecta direto com o módulo de alerta de doença por região	Não (specialty/notes são texto livre)
Prioridade legal de atendimento (idoso 60+, gestante, PCD, lactante)	Define ordem de fila, obrigatório por lei (Lei 10.048/2000)	Não — todo pendente é tratado igual
Nome do responsável/acompanhante	Rotina real de posto (menor de idade, dependente)	Não
Etnia/raça-cor (autodeclaração)	Campo padrão CADSUS/e-SUS, indicador de equidade racial em saúde	Não
Histórico vacinal / comorbidades (CIAP2)	Parte do prontuário básico e-SUS AB	Não

CPF, RG, nome da mãe, endereço com zona urbana/rural, e `blood_type` já estão alinhados com CADSUS — não é retrabalho, é complemento. Prioridade dentro de E12 (do mais simples/alto impacto ao mais complexo):

1. **Nome social e prioridade legal de atendimento** — campo simples, alto impacto de conformidade legal, esforço baixo.
2. **CNS** — campo de texto validado (15 dígitos, algoritmo de dígito verificador público), sem integração externa na primeira versão (só captura e valida formato).
3. **Nome do responsável/acompanhante** — condicional a paciente menor de idade.
4. **Etnia/raça-cor** — select fechado com as categorias oficiais do IBGE/CADSUS.
5. **CID-10 estruturado e integração real com RNDS/e-SUS** — maior esforço, depende de tabela CID completa ou serviço externo; tratar como item de pesquisa técnica separado, não obrigatório para o primeiro lançamento comercial.

5. Sprints (2 semanas cada)

Cada sprint tem: objetivo, escopo, Definition of Done (DoD). O DoD é sempre cumulativo — cada sprint exige que tudo do DoD anterior continue verdadeiro.

Sprint 0 — Fundação e entendimento do que já existe (2 semanas)

Objetivo: todo o time entende o problema, o público, e a base de código existente antes de tocar em qualquer linha nova.

- PO e time revisam o PDF original e traduzem em linguagem de produto (este documento).
- Time lê e roda o projeto localmente (README já documenta o setup).
- Time percorre as 5 telas como se fosse cada um dos 3 papéis de usuário (paciente, atendente, admin).
- Backlog inicial (seção 4) validado com o time; PO prioriza.

DoD: todo integrante do time consegue explicar, sem consultar nada, o que o app faz e para quem; ambiente local rodando na máquina de cada um.

Sprint 1 — Fundação de segurança básica (2 semanas)

Objetivo: fechar os riscos críticos de segurança antes de qualquer outro trabalho (E1).

- Girar `JWT_SECRET` e senha MySQL reais; garantir que `.env` nunca é commitado.
- `git init` no projeto (hoje sem controle de versão) + primeiro commit limpo.
- Criar `server/.env.example` real.
- Adicionar rate limiting em `/auth/login` e `/auth/signup`.
- Adicionar `helmet` (cabeçalhos HTTP de segurança).
- Corrigir fallback de CORS (`origin: '*'`) para bloqueio explícito em produção.

DoD: nenhum segredo real no repositório git; login resiste a tentativa de força bruta trivial; `git log` mostra histórico real do projeto a partir daqui.

Sprint 2 — Validação de dado ponta a ponta (2 semanas)

Objetivo: back-end para de confiar cegamente no front-end (E2).

- Validação de payload no servidor (email, CPF com dígito verificador, telefone, datas) em todas as rotas `POST / PUT / PATCH`.
- Igualar obrigatoriedade de campo entre front e back no cadastro (CPF, RG, nascimento, gênero, nome da mãe, celular).
- `scheduled_at` (consulta/transporte/exame) validado como não-passado também no servidor.
- `job_rol` (cargo do atendente) e `region` (alerta de doença) viram `<select>` fechado com opção "Outro" condicional, em vez de texto livre.

DoD: chamar a API diretamente (sem passar pela tela) não permite gravar dado inválido — testado manualmente com Postman/Insomnia pelo próprio time.

Sprint 3 — Alinhamento com padrão de dado do SUS (2 semanas)

Objetivo: cadastro de paciente passa a usar os campos que a rede pública de saúde real usa, viabilizando conversa técnica séria com a prefeitura (E12, ver seção 4.1).

- Adicionar `social_name` (nome social) ao schema e ao wizard de cadastro, exibido no lugar do nome legal quando preenchido.
- Adicionar campo de **prioridade legal de atendimento** (idoso 60+, gestante, PCD, lactante) — calculado automaticamente por idade quando possível (ex.: 60+), com opção de marcação manual para os demais casos; usado para ordenar fila de atendimento no painel do atendente.
- Adicionar `cns` (Cartão Nacional de Saúde, 15 dígitos) ao cadastro, com validação de formato e dígito verificador (função pura, sem serviço externo) — opcional no primeiro momento (paciente pode não ter em mãos no cadastro), mas capturado sempre que disponível.
- Adicionar `guardian_name` (nome do responsável/acompanhante), condicional a paciente menor de 18 anos calculado a partir de `birth_date`.

- Adicionar `race_color` (etnia/raça-cor, autodeclaração) como `<select>` fechado com as categorias oficiais do IBGE/CADSUS (branca, preta, parda, amarela, indígena, prefiro não informar).
- **Fora de escopo desta sprint, registrado para pesquisa técnica futura:** CID-10 estruturado e integração com RNDs/e-SUS AB — maior complexidade, não bloqueia o lançamento comercial inicial.

DoD: cadastro de paciente captura nome social, CNS (quando informado), prioridade legal e responsável (quando aplicável); painel do atendente ordena fila considerando prioridade legal, não só ordem de chegada.

Sprint 4 — Acessibilidade e ações críticas (2 semanas)

Objetivo: interface segura e usável para o público-alvo real (E3).

- Substituir todo `alert()` / `confirm()` / `prompt()` nativo por modal do próprio design system.
- Reagendamento (admin e atendente) troca `prompt()` de data digitada por `<input type="datetime-local">`.
- `aria-invalid` + mensagem inline nos campos obrigatórios do wizard de cadastro.
- Navegação por teclado (seta esquerda/direita) nas abas (`role="tab"`).
- Loading state nos botões de agendamento (evita duplo clique/duplo envio).

DoD: nenhuma ação destrutiva (cancelar consulta, mudar status de conta) depende de `confirm()` nativo; wizard de cadastro navegável 100% por teclado.

Sprint 5 — Consolidação de design system e código (2 semanas)

Objetivo: reduzir duplicação que já foi identificada como risco de manutenção (E4).

- Extrair `style="..."` inline repetido para classes utilitárias em `styles.css`.
- Extrair helpers JS duplicados (`switchTab` , `statusClass` , `stat` , `levelClass`) para `assets/ui.js` compartilhado.
- Consolidar lógica quase idêntica de `appointments` / `transports` / `exams` no backend (`me/*` , `admin` , `staff`) em um helper comum.
- Padronizar padrão "Outro" condicional em todos os selects fechados (`exam_type` , `location`).

DoD: qualquer correção de bug em um dos 3 papéis (paciente/admin/atendente) é feita em um lugar só, não em 3 cópias.

Sprint 6 — Testes automatizados e CI (2 semanas)

Objetivo: parar de depender só de teste manual (E5).

- Escolher framework de teste (recomendado: `vitest` ou `jest` — ambos JS puro, sem sair da linguagem do projeto).
- Cobrir com teste automatizado: login, cadastro, criação/cancelamento de agendamento, controle de acesso por papel (`verifyToken` , `requireAdmin` , `requireStaff`).
- Configurar CI simples (GitHub Actions) rodando os testes a cada push.

DoD: pipeline de CI verde bloqueando merge se teste quebrar; cobertura mínima nas rotas de autenticação e agendamento.

Sprint 7 — Deploy real e ambiente de produção (2 semanas)

Objetivo: sistema sai da máquina local e vai para um ambiente acessível de verdade (E6).

- Escolher hospedagem (ex.: um provedor com camada gratuita/barata para o backend Node + MySQL gerenciado; frontend estático pode ir para qualquer CDN/hosting estático).
- HTTPS obrigatório.

- Variáveis de ambiente de produção configuradas fora do código.
- Monitoramento básico de disponibilidade (E11 parcial: saber se o sistema caiu).

DoD: URL pública acessível, com HTTPS, rodando a versão mais recente aprovada; time sabe (recebe aviso) se o sistema cair.

Sprint 8 — LGPD e consentimento (2 semanas)

Objetivo: conformidade legal mínima para operar com dado de saúde real (E7).

- Texto de consentimento explícito no cadastro, explicando finalidade de cada dado sensível coletado (CPF, RG, dado de saúde, CNS, etnia/raça-cor — este último é dado sensível por definição na LGPD, art. 5º, II).
- Política de privacidade real (já existe `privacy.html` — revisar e completar).
- Definir e documentar política de retenção/expurgo de dado.
- Confirmar com a Secretaria de Saúde se já existe termo de consentimento próprio a reaproveitar.

DoD: nenhum dado sensível é coletado sem texto de consentimento visível; política de privacidade completa e acessível a partir de toda tela pública.

Sprint 9 — Lembretes automáticos (2 semanas)

Objetivo: entregar a funcionalidade prometida na proposta original, ainda pendente (E8).

- Job agendado no backend que gera lembrete automático 1 dia antes de consulta/transporte/exame.
- Exibição de lembrete não lido em destaque no dashboard do paciente.

DoD: paciente com consulta marcada para amanhã vê lembrete automático sem ação manual de ninguém.

Sprint 10 — Preparação comercial: multi-tenant leve (2 semanas)

Objetivo: viabilizar revenda para outro município sem reescrever o sistema (E9).

- Introduzir conceito de "unidade gestora" (município/rede) na base de dado — unidades de saúde, alertas de doença e usuários passam a pertencer a uma unidade gestora.
- Paineal admin restrito aos dados da própria unidade gestora.
- Documentar processo de onboarding técnico de um novo cliente (novo município).

DoD: é possível cadastrar uma segunda prefeitura de teste no mesmo sistema sem um cliente ver dado do outro.

Sprint 11 — Material comercial e onboarding (2 semanas)

Objetivo: transformar o sistema pronto em algo vendável para gestor público (E10).

- Apresentação/demo gravada ou ao vivo para não-técnico (gestor de saúde não vai ler código).
- Documentação de "como adotar o Saúde+ na minha cidade" (passos, requisitos, custo estimado).
- Página pública de apresentação do produto (pode reaproveitar/evoluir `index.html`).

DoD: uma pessoa fora do time consegue entender o valor do produto e o próximo passo para contratar, sem precisar de explicação verbal do time.

Sprint 12 — Estabilização e lançamento (2 semanas)

Objetivo: fechar pendência residual, correr última rodada de teste com usuário real, lançar.

- Teste com usuário real (morador voluntário + atendente do posto), replicando o "Mês 4" da proposta original — mas agora validado por sprint, não só um mês solto no fim.
- Correção de bugs encontrados no teste.

- Divulgação em posto de saúde, escola, rede social (conforme proposta original, "Mês 5").

DoD: sistema em produção, sem bug crítico aberto, com pelo menos 1 unidade de saúde e 1 grupo real de pacientes usando de verdade.

Nota honesta de PO: 13 sprints (26 semanas / ~6,5 meses) é uma estimativa realista para um time de adolescentes iniciantes com apoio de IA, não uma garantia. Sprints podem render menos que o planejado enquanto o time ainda está aprendendo o básico (Sprint 0-2, especialmente) — é esperado e normal replanejar o backlog nas retrospectivas, não é fracasso.

6. Critérios de "pronto pra mercado"

Fechar as sprints acima não é o mesmo que estar pronto para vender. Isso é deliberado no enunciado deste plano — abaixo, o que separa "funciona" de "profissional o suficiente para o mercado de trabalho e para cobrar de um cliente":

- **Segurança validada por terceiro:** idealmente uma revisão de segurança externa (mesmo que informal, ex.: professor de tecnologia, voluntário da área) antes do primeiro cliente pagante.
- **Contrato e suporte:** definir o que acontece quando o sistema cai às 22h de uma sexta — quem responde, em quanto tempo (mesmo que informal para o primeiro cliente).
- **SLA mínimo declarado:** mesmo que simples ("respondemos em até 24h úteis"), a prefeitura precisa saber o que esperar.
- **Backup e recuperação de desastre:** dado de saúde perdido por falha de banco sem backup é inaceitável — testar restauração de backup pelo menos uma vez antes do lançamento.
- **Precificação:** decisão de negócio (fora do escopo técnico deste documento) mas precisa existir antes da Sprint 11.
- **Feedback loop pós-lançamento:** canal simples (formulário, WhatsApp, e-mail) para usuário real reportar problema — sem isso, o time não escala além da Sprint 12.

Este documento não fecha esses pontos de negócio (preço, contrato formal, CNPJ/formalização) — eles ficam registrados aqui como lacuna consciente, não esquecimento, para o PO tratar fora do backlog técnico.

7. Riscos e mitigação

Risco	Impacto	Mitigação
Time iniciante pode subestimar esforço de cada item do backlog	Sprints atrasam	Scrum Master ajusta escopo a cada planning; velocity real (não a estimada) guia o sprint seguinte
Dependência de IA sem entendimento pode gerar código que ninguém do time sabe explicar	Bug impossível de corrigir depois	DoD de todo item exige que ao menos 1 pessoa explique a lógica, não só "a IA gerou"
Dado de saúde de pessoas reais em ambiente ainda em amadurecimento	Risco legal e de confiança pública	Sprint 8 (LGPD) não é opcional nem adiável para depois do primeiro uso com dado real

Equipe extraescolar (tempo limitado, calendário escolar)	Cadência de 2 semanas pode não ser sustentável o ano todo	Revisar cadência no fim de cada bloco de 3 sprints (retrospectiva "de retrospectivas")
Ambição comercial pode inflar escopo além do que o time consegue sustentar sozinho	Produto lançado sem suporte real	Sprint 11 é honesta sobre o que é MVP comercial vs. visão de longo prazo — não vender promessa que o time não consegue cumprir sozinho

8. Apêndice — Achados detalhados do diagnóstico técnico

Este apêndice condensa os dois diagnósticos completos já produzidos sobre a base de código existente — [PLANO MELHORIAS.md](#) (backend, segurança, dado) e [PLANO UIUX.md](#) (interface, acessibilidade) — para que este documento seja autocontido. Cada achado remete ao épico correspondente na seção 4.

8.1 O que já está bem resolvido (não mexer sem motivo)

- SQL 100% parametrizado (`?` bindings) em todas as rotas — sem risco de injeção SQL.
- Isolamento por `user_id` consistente em rotas `/me/*`.
- `password_hash` nunca retornado pela API; CPF/RG restritos a admin.
- Log de auditoria (`audit_log`) em ações administrativas.
- Separação clara de middleware (`verifyToken` , `requireAdmin` , `requireStaff`).
- Design system consolidado em `styles.css` via CSS custom properties, paleta com propósito declarado (alto contraste, tipografia generosa para público com baixa familiaridade digital).
- Acessibilidade de base já presente: `:focus-visible` global, `prefers-reduced-motion` respeitado, `aria-selected` / `role="tab"` / `role="tabpanel"` nas abas dos 3 painéis internos.
- Sanitização de output (`esc()`) usada consistentemente antes de injetar dado dinâmico no DOM — evita XSS refletido.
- Estado vazio tratado, feedback de carregamento (spinner) e de ação (toast) presentes em todo fluxo assíncrono.
- Wizard de cadastro em 4 passos com indicador visual de progresso, máscaras de input em tempo real (CPF, telefone, CEP) e autopreenchimento via ViaCEP.

8.2 Segurança — crítico (backend)

1. `.env` real com senha de banco, `JWT_SECRET` e senha de admin em texto plano solto no disco — risco de ser commitado no primeiro `git init` (endereçado na Sprint 1).
2. Sem rate limiting em `/auth/login` — força bruta de senha é trivial (Sprint 1).
3. JWT em `localStorage` sem blacklist/revogação — token vazado por XSS fica válido até expirar (trade-off aceito, documentado na seção 3.4).
4. Sem Helmet/cabeçalhos de segurança HTTP (Sprint 1).
5. CORS com fallback `origin: '*'` quando `CORS_ORIGIN` não definido (Sprint 1).
6. Sem validação de formato de entrada (email, CPF, telefone, datas) além de checagem de presença (Sprint 2).
7. Sem guia de deploy/HTTPS documentado (Sprint 7).

8.3 Robustez e correção — alto (backend)

8. Sem testes automatizados — nenhuma regressão é pega automaticamente (Sprint 6).
9. Sem `server/.env.example` real no repositório (Sprint 1).
10. Rotas de agendamento (`appointments` , `transports` , `exams`) duplicam lógica quase idêntica entre `me/*` , `admin` e `staff` — 3 cópias do mesmo padrão (Sprint 5).

11. Lembretes são somente leitura, sem geração automática (Sprint 9).
12. Sem paginação real (`OFFSET /cursor`) além do `LIMIT` fixo — dado além do limite fica inacessível pela UI.
13. Erro handler global sempre devolve 500 genérico; erros de constraint do MySQL não são traduzidos em mensagem útil.
14. Mudança de `role` de usuário só é possível direto no banco — decisão de segurança intencional, mas sem trilha de auditoria própria.

8.4 Manutenibilidade e DX — médio (backend)

15. Sem linter/formatter configurado (`.eslintrc` , `prettier`).
16. Sem CI configurado (Sprint 6 resolve).
17. Sem tipagem (JS puro, sem TypeScript/JSDoc consistente) em schema com 20+ campos em `users` .
18. `assets/api.js` é um único arquivo global (IIFE), sem módulos ES.
19. Sem `package.json` no nível raiz do projeto (só em `server/`).
20. Datas tratadas como string livre (`scheduled_at`) sem checagem de formato/futuro no servidor (Sprint 2).

8.5 Recebimento de dados e formulário de cadastro (achado central)

O front valida mais do que o back exige — toda a integridade do cadastro hoje depende do JavaScript do navegador, não do servidor. Qualquer chamada direta à API contorna essas regras (endereçado na Sprint 2).

- `job_role` é texto livre, gerando inconsistência ("Enfermeira", "enfermeiro", "ENFERMEIRA(O)") que quebra filtro/relatório por cargo.
- CPF e RG são obrigatórios no HTML mas opcionais na API (`|| null`); mesma lacuna em `birth_date` , `gender` , `mother_name` , `mobile` .
- Confirmação de senha só é checada no cliente, nunca no servidor.
- CPF sem dígito verificador — `111.111.111-11` passa em front e back.
- Email sem validação real no back (`a@b` passa).
- Celular sem validação de DDD/tamanho real.
- Datas de agendamento sem checagem de "não pode ser no passado" no servidor.
- CEP preenche endereço mas campos continuam editáveis sem revalidação de consistência.
- `zone` / `reference_point` obrigatórios para todos, mesmo quem já preencheu endereço urbano completo.
- Validação do wizard é só "forward" — reenvio pelo passo 4 não revalida os passos anteriores.
- Nenhum campo sensível explica *por que* é coletado (transparência LGPD).

8.6 Recebimento de dados — telas internas (dashboard, admin, atendente)

- `transport_type` e `exam_type` não têm campo livre condicional para "Outro" (ao contrário de `specialty` , que já tem esse padrão) — dado se perde quando "Outro" é selecionado (Sprint 5).
- Catálogo de unidades de saúde (`health_units`) não tem tela de cadastro/manutenção pelo admin — exige `UPDATE` manual no banco.
- Campo `notes` / `reason` sem `maxlength` visível.
- Reagendamento usa `prompt()` nativo pedindo data digitada à mão, sem validação de formato no front nem no back — replicado em admin e atendente (Sprint 4).
- `region` do alerta de doença é texto livre, mesmo problema de `job_role` (Sprint 3 normaliza via padrão CADSUS/select fechado).
- Duas telas de busca de usuário (Histórico vs. Usuários) fazem quase a mesma coisa com UX diferente (Sprint 5 unifica).
- Alteração de status de conta não pede motivo/observação para o audit log.
- Busca de paciente pelo atendente só retorna contas `ativo` — paciente inativo/pendente "some" sem explicação, risco de cadastro duplicado.

8.7 Usabilidade e acessibilidade — alto impacto (UI/UX)

1. `confirm()` nativo para cancelar agendamento — inconsistente com o design system, ruim para leitor de tela (Sprint 4).
2. `alert()` / `confirm()` como única confirmação de ação destrutiva, sem explicar consequência específica (Sprint 4).
3. Sem loading state nos botões de submit de `addTransport` / `addAppt` / `addExam` — permite duplo envio acidental (Sprint 4).
4. Mensagens de erro genéricas demais — não diferenciam erro de rede, validação ou servidor.
5. Navegação por teclado (seta esquerda/direita) não implementada nas tabs (Sprint 4).
6. Campos obrigatórios do wizard não usam `aria-invalid` nem mensagem inline associada (Sprint 4).
7. Sem indicação de força de senha no cadastro.

8.8 Consistência e polimento — médio impacto (UI/UX)

8. Duplicação de CSS inline (`style="..."`) espalhada pelos HTMLs em vez de classes utilitárias (Sprint 5).
9. Duplicação de JS entre `dashboard.html` , `admin.html` , `atendente.html` (`switchTab` , `statusClass` , `helpers`) (Sprint 5).
10. Formulários de agendamento não usam `max` em `datetime-local` , só `min` — permite agendar décadas no futuro sem aviso.
11. Botão "Editar" faz scroll ao formulário mas não destaca visualmente o "modo edição ativo".
12. Padrão "Outro" condicional existe para especialidade mas não para tipo de exame/local (Sprint 5).
13. Busca de usuário e busca de histórico são telas separadas fazendo a mesma coisa (Sprint 5).
14. Sem botão "Cancelar" explícito nos formulários de criação (só "Cancelar edição").
15. `.wizard-steps` sem regra de mobile específica para telas muito pequenas (< 360px).

8.9 Refinamento — baixo impacto (UI/UX)

16. Fonte Inter carregada via Google Fonts, sem hospedagem local — relevante para público rural com internet instável.
17. Ícones SVG inline repetidos sem sprite/symbol compartilhado.
18. Sem skeleton loading nas listas (usa spinner central genérico).
19. Sem dark mode (ausência consciente, baixa prioridade dado o público).
20. Contraste de `.opt` ("(opcional)") deve ser confirmado com ferramenta de contraste real (AA).
21. Emoji de aviso (🚨) sem `aria-label` / `role="img"` .

9. Referências

- Proposta original da equipe: [Conectados para o Sucesso 2.0.pdf](#)
- Diagnóstico técnico backend/segurança/dado: [PLANO MELHORIAS.md](#)
- Diagnóstico técnico UI/UX/acessibilidade: [PLANO UIUX.md](#)
- Setup e decisões técnicas herdadas: [README.md](#)
- Roteiro de teste com aluno/usuário real: [ROTEIRO_TESTE_ALUNOS.md](#)